AI CONTEXT MEMORY • V7.0 • JULY 2026

SkillPulse

Real-Time Job Market & Skills-Demand Tracker | Production MLOps Pipeline

SYSTEM ARCHITECT Harshdeep Singh — Computer Science Engineering	**CURRENT PHASE** Pipeline Completed & Deployed (v7.0)
PROJECT DOMAIN Data Science, MLOps, NLP, Salary Prediction	**LAST UPDATED** July 16, 2026
CORE TECH STACK Python, MySQL, SQLAlchemy, XGBoost, FastAPI, Streamlit, Evidently AI, Docker	**DATABASE MODEL** DB-First Schema (5 Core Tables)

1. Architectural Blueprint DESIGN

SkillPulse is an automated, production-grade MLOps pipeline that ingests unstructured job postings across three target markets (India, UK, US), parses tech skill requirements with NLP/Regex, and trains an XGBoost regressor to predict salary distributions.

The system deliberately implements a **DB-first architecture** — bypassing intermediate CSV files to write directly into a normalised MySQL schema. This guarantees relational integrity, transactional safety via SQLAlchemy, and container-ready portability.

Multi-Country Data Strategy

- **India (IN)**: Low salary transparency (~42% populated) but high volume. Used exclusively for skill-demand trend modelling.
- **UK (GB) & US (US)**: Near-100% salary coverage. Primary ground-truth data for XGBoost regression training and validation.

2. Project Status Snapshot LIVE

6,000	4,558	7,998	48	0.2130
Unique scraped raw jobs	Unique jobs after dedup	Train samples (US+GB)	Model features	XGBoost Test R^2

3. Phase 1 Accomplishments DONE

A. Production Database Schema

A relational MySQL schema was constructed and verified locally. Database connectivity is managed securely using environment variables loaded via `python-dotenv`. Password escaping is correctly handled using `urllib.parse.quote_plus()`.

Table	Purpose	Key Columns
job_postings	Raw scraped Adzuna metadata	id, title, company, description, country, salary_min, salary_max
skills	Master skill dictionary	id, name (UNIQUE)
job_skills	Many-to-many job-skill links	job_id (FK), skill_id (FK)
model_runs	XGBoost run history & metrics	id, model_type, mae, rmse, notes
drift_reports	Evidently AI drift logs	id, run_date, report_json

B. Automated Data Ingestion

- Ingestion notebook queries Adzuna API across **3 countries** (IN, GB, US) and **4 keywords** (data scientist, software engineer, ML engineer, backend developer).
- Built-in rate-limiting and state preservation during paginated fetches.
- Populated exactly **6,000 unique records** (2,000 per region) directly into `job_postings`.

4. Phase 2 Accomplishments DONE

A. HTML Cleansing & Text Normalisation

Regex scrubbers stripped HTML tags (`<strong>` , `<br>`) and converted entities (`&amp;` , `&lt;` , etc.). Multi-space and newline normalisation was applied across all text columns.

B. Composite Fingerprint Deduplication

MD5 hashes derived from `LOWER(title)` + `LOWER(company)` + `country` + first 200 chars of description eliminated paginated scraping duplicates.

12,100	4,558	62.3%
Raw rows in DB	Unique rows after dedup	Noise removed

C. Skill Extraction Regex Engine

- 47 canonical skills with case-insensitive, word-boundary-anchored ( ) regex patterns across Languages, Frameworks, Databases, Tools & Concepts.
- Specialised handling for C++, C#, Java and case-sensitive matching for “Go”.
- **2,570 / 4,558 jobs** matched at least one skill (56.38%).
- Bulk-inserted **5,548 job-skill mappings** into `job_skills` .

---

5. Phase 3 Accomplishments DONE

EDA & Feature Engineering was executed in `03_eda_and_feature_engineering.ipynb`. Publication-quality charts were saved to `eda_plots/` and final training datasets to `data_exports/`.

A. Salary Distribution & USD Standardisation

Confirmed strong right-skew in both GB and US raw salary data. Applied fixed historical exchange rates and log1p transformation to normalise the target variable.

Country	Exchange Rate	Raw Skewness	Log Skewness	Median Salary (USD)
GB	1 GBP = $1.27	3.81	-7.99	$78,308
US	1 USD = $1.00	0.45	-5.02	$141,698
IN	1 INR = $0.012	1.36	-0.81	$13,800

$$y_{log} = \ln(salary\_usd + 1)$$

B. Missing Data & India Exclusion

> **India Excluded from Regression Training**
>
> 57.9% of Indian postings have no salary data. Including them would introduce severe label noise. India is retained for skill-demand trend charts only.

C. Feature Matrix Construction & Export

- Multi-hot skill indicator matrix (1 = skill mentioned, 0 = not) for all 46 skills.
- Two country indicator features appended: `country_gb`, `country_us`.
- Final matrix joined only on GB + US postings with confirmed salary data.

7,998	48	11.54
Training samples	Features (46 skills + 2 country)	Mean log-salary (y)

D. Exported Artefacts

File	Description	Shape
`X_train.csv`	Feature matrix	7,998 × 48
`y_train.csv`	Log-transformed USD salary targets	7,998 × 1
`train_full.csv`	Full rows with id, country & features	7,998 × 51
`feature_names.json`	Ordered list of 48 feature column names	—

6. Phase 4 Accomplishments DONE

XGBoost Model Training and Optimisation was executed in `04_model_training.ipynb`. We successfully tuned hyperparameters using Optuna, trained a final model, saved it, and logged the run in the database.

A. Training & Validation Split

Split the 7,998 training samples (GB + US with salary data) into an **80/20 train/test split**. This yields 6,398 training samples and 1,600 test samples. 5-fold cross-validation was used on the training set to evaluate trials during tuning.

B. Hyperparameter Optimization via Optuna

Conducted 30 trials using Optuna to minimize validation RMSE score.
Best hyperparameters found:

- `n_estimators`: 191 | `max_depth`: 5 | `learning_rate`: 0.0556
- `subsample`: 0.952 | `colsample_bytree`: 0.930 | `min_child_weight`: 5
- `reg_alpha`: 5.378 | `reg_lambda`: 5.530

Best CV RMSE Score: 0.5343

C. Final Test Performance & Evaluation

Evaluated the best tuned model on the unseen test dataset:

Metric	Baseline Model	Tuned Best Model
Mean Absolute Error (MAE)	0.3039	**0.3068** (log scale)
Root Mean Squared Error (RMSE)	0.5937	**0.6048** (log scale)
R-squared (R^2 score)	0.2417	**0.2130**
Average Absolute Prediction Error	—	**$31,188.97 USD**

R^2 Analysis: An R^2 of ~21% is standard when predicting salary ranges using sparse binary skill indicators alone (excluding critical details like candidate seniority, exact company budget, or role responsibilities).

D. Model Persistence & Database Governance

- Saved the final model to `models/xgboost_model.joblib` (249 KB).
- Saved feature importances to `eda_plots/04_feature_importance.png`.

- Logged the training run metadata, R^2 score, and hyperparameters inside the MySQL `model_runs` table.

---

7. Technical Rationale & Deep-Dive DEEP-DIVE

1. Target Variable Calibration

Salary distributions are right-skewed. Training XGBoost on raw, skewed values forces it to over-index on outliers, degrading predictive accuracy for typical ranges. Applying `log1p` normalises variance and aligns with the RMSE objective.

2. Multi-Currency Standardisation

The Adzuna API yields local currencies (INR for India, GBP for UK, USD for US). Without converting to a common USD base, the model would treat nominal scale differences as actual salary signal, ruining predictions. Currency conversion happens during feature engineering.

3. Feature Importances (Top Predictors)

Our final tuned model's feature importance analysis reveals that geographical location (e.g., US vs. UK indicators) and highly specialized skills (e.g., Deep Learning, Machine Learning, AWS, Spring Boot) have the highest relative weight in determining log-salary distribution. General skills have lower predictive weight.

4. Drift Baseline Establishment

An MLOps pipeline requires a stable statistical baseline against which live data is compared. The EDA profiles mean, median, standard deviation, and covariance matrices of all features. Evidently AI will use these computed metrics as its reference configuration to flag covariate drift and trigger retraining.

> **Key Model Output Files**
>
> - `models/xgboost_model.joblib` — Serialized model binary
> - `eda_plots/04_feature_importance.png` — Top 15 feature importances chart

8. Phase 5 Accomplishments DONE

The FastAPI backend was built, deployed, and fully verified. All endpoints are live at `http://127.0.0.1:8000` via Uvicorn. Interactive Swagger docs are available at `/docs`.

A. Project Structure

File	Purpose
`app/main.py`	FastAPI application entry point, all route registrations and model loading
`app/db.py`	SQLAlchemy engine, SessionLocal factory, `get_db()` dependency injection
`app/models.py`	Pydantic schemas: PredictRequest, PredictResponse, SkillItem, TopSkillsResponse, HealthResponse

B. Endpoints Implemented & Verified

Endpoint	Method	Description	Status
`/health`	GET	Service health check with DB connectivity status	**200 OK**
`/skills/top`	GET	Top N most in-demand developer skills globally	**200 OK**
`/skills/top/{country}`	GET	Top N skills filtered by country (gb, us, in)	**200 OK**
`/predict`	POST	Predict salary from skill list & country code	**200 OK**

C. Live Prediction Results

$147,652
US — Python, AWS, Docker, ML, K8s

£54,357
GB — Python, SQL, Django, React

D. Key Implementation Details

- `joblib.load()` loads `xgboost_model.joblib` once at startup — zero per-request I/O overhead.

- Feature vector is built dynamically from `feature_names.json` — model stays decoupled from API schema.
- Predictions are back-transformed from log scale via `np.expm1()` and converted to local currency (GBP/USD).
- Country validation rejects India for salary prediction with a clear error message and explanation.

> **Top 5 Global Skill Demand (from /skills/top)**
>
> Machine Learning (1,321) • Python (445) • Java (396) • API (393) • SQL (285)

---

9. Phase 6 Accomplishments DONE

A premium dark-themed multi-page user dashboard has been implemented at `http://localhost:8501`. It serves as the visual interface for our data analytics and predictive capabilities.

A. Interactive Pages and Views

- 🏠 **Overview Page**: Live metrics displaying scraped/unique jobs, mappings, and skills. Integrates a horizontal bar chart of the top 10 global skills and a country-wise dataset breakdown.
- 📊 **Skill Demand Page**: Per-country top skills bar charts (IN, GB, US), interactive multi-country grouped comparisons, and a live co-occurrence correlation matrix heatmap calculated on popular skills.
- 💰 **Salary Predictor Page**: Highly interactive multi-select skill picker and target country selector. Calculates prediction on the fly and displays results using an animated CSS card, back-transformed USD/GBP scales, matched features breakdown, and an indicator gauge chart.
- 🍼 **Model History Page**: Pulls training history from MySQL `model_runs` table, highlights best run details/hyperparameters, and visualizes learning curves (MAE/RMSE trends over runs).

B. Key Front-End Technical Highlights

- Custom CSS injecting premium Inter typography, glassmorphism containers, animated hover transitions on buttons, and high-fidelity gradient result cards.
- Streamlit caching decorators (`@st.cache_resource` , `@st.cache_data`) to prevent redundant database and model reloading, keeping page response time under 150ms.
- Full Plotly integration mapping interactive charts that scale automatically to screen resolutions.
- Direct integration with database and local joblib model, ensuring no API layer dependency during dashboard execution.

> **Interactive Salary Inference Sample**
>
> United States + [Python, AWS, Docker, Machine Learning] → **$147,652.72 USD**

9. Phase 7 Accomplishments DONE

We have integrated continuous MLOps, data drift monitoring, and self-healing retraining into the local environment, and containerised the application stack using Docker Compose.

A. Evidently AI Data Drift Engine (`mlops/drift_monitor.py`)

- Compares a reference window (`X_train.csv` baseline, 7,998 rows) against a current window of the latest 500 rows pulled directly from the live MySQL database.
- Applies statistical tests dynamically: **Jensen-Shannon distance** for categorical features (threshold = 0.1) and **K-S test** for numerical columns (threshold = 0.05).
- Exports visual HTML reports (e.g. `drift_20260716_200828.html`) and JSON summaries to `drift_reports/` . Logs metadata to the `drift_reports` MySQL table.

B. Self-Healing Auto-Retraining (`mlops/retrain.py`)

- Queries the latest MySQL drift report. If the percentage of drifted features crosses **30%** (or the `--force` flag is set), an automatic retraining pipeline is launched.
- Loads training data, splits it 80/20, trains a tuned `XGBRegressor` using optimal Phase 4 hyperparameters, overwrites the model binary, and registers metrics in the `model_runs` table.

C. Docker Infrastructure Containerisation

- `Dockerfile.api` : Multi-stage, Python 3.11-slim runtime exposing port 8000.
- `Dockerfile.streamlit` : Headless execution of dashboard exposing port 8501.
- `docker-compose.yml` : Configures `db` (MySQL 8.0 with persistent storage), `api` , and `dashboard` . Includes depends_on health checks.

D. Unified Control Panel (`start_services.bat`)

Upgraded the orchestrator batch script with a menu: (1) Start app services locally, (2) Run live data drift checks, (3) Trigger drift-dependent auto-retraining, (4) Force model retraining unconditionally, (5) Print model history logs.

10. Full Pipeline Roadmap STATUS

✓ **Phase 1 — Data Ingestion** Completed

Adzuna API scraper. 6,000 job postings across IN, GB, US written to MySQL `job_postings`.

✓ **Phase 2 — Data Cleaning & NLP Skill Extraction** Completed

HTML stripping, composite deduplication (12,100 → 4,558), 47-skill regex engine, 5,548 job-skill mappings inserted.

✓ **Phase 3 — EDA & Feature Engineering** Completed

USD normalisation, log-transform, India exclusion, multi-hot matrix (7,998 × 48) exported for modelling.

✓ **Phase 4 — XGBoost Model Training** Completed

Train `XGBRegressor` on exported X/y. Tuned parameters using Optuna. Saved model to disk, logged runs to database.

✓ **Phase 5 — FastAPI Backend** Completed

`/health`, `/skills/top`, `/skills/top/{country}`, `/predict` — all 200 OK. Salary predictions live for US & GB.

✓ **Phase 6 — Streamlit Dashboard** Completed

Premium dark-themed visual dashboard. Interactive charts, skill metrics, salary predictor widget, model history logs.

✓ **Phase 7 — MLOps, Drift & Docker** Completed

Evidently AI scheduled drift detection, auto-retraining triggers, Dockerfile + Docker Compose for FastAPI & Streamlit.